Technical White Paper

The Anchor Protocol:

A Formally Verified Control Plane for Local Agent Swarms

Erich Owens

Curiositech LLC

engineering@portdaddy.dev

May 2026
Version 1.2 (Port Daddy v3.13.0)

Abstract

As AI agents transition from isolated copilots to collaborative, autonomous swarms, local development environments face a crisis of coordination and trust. Existing tools designed for human-driven, static infrastructures lack the primitives required to prevent dynamic agents from corrupting shared state or squatting on ephemeral ports.

This paper introduces the **Anchor Protocol**, a cryptographic and semantic identity framework built into the Port Daddy daemon. We detail the protocol's evolution from symmetric MACs to asymmetric Ed25519 signatures and multi-hop delegation chains (inspired by Macaroons [4]). Furthermore, we present the formal verification of the protocol's design using the **ProVerif** symbolic analyzer [1] and the mathematical proof of its implementation, demonstrating immunity to algorithm confusion [10], impersonation, and timing side-channels.

Keywords: distributed systems security, formal verification, capability-based security, multi-agent coordination, symbolic protocol analysis, ProVerif, JWT security, Ed25519

Reading time: approximately 30 minutes end-to-end. The companion paper, The Bonded Commons: Agent Transactions and the Federated Sovereign *(Owens 2026), carries the economic and governance argument; either paper can be read first.*

Contents

1 Introduction: The "Local Swarm" Problem

In a multi-agent development environment, agents operate concurrently to read files, spin up test servers, and execute commands. This introduces three critical threat vectors on `localhost`:

1. **Port Squatting (The "Ghost in the Harbor"):** If Agent A crashes, a malicious or malfunctioning Agent B can bind to Agent A's previously assigned port, intercepting traffic meant for A.
2. **Resource Contention:** Agents fighting over shared resources (e.g., a SQLite database file) without a centralized locking mechanism cause state corruption.
3. **Privilege Escalation:** A "Reviewer" agent delegated by a "Coder" agent should not possess the rights to initiate production deployments, yet local environments rarely enforce principle-of-least-privilege between processes.

To solve this, Port Daddy introduces the concept of a **Harbor**—a zero-trust, namespace-isolated execution environment where agents must prove their identity and capabilities. Our approach builds upon foundational work in capability-based security by Dennis and Van Horn [7] and the principle of least privilege articulated by Saltzer and Schroeder [15].

Companion paper. This paper is the cryptographic substrate. Its companion, *The Bonded Commons: Why Free-Market Agent Economies Decay and How To Fix Them*, builds the economic and game-theoretic layer on top: bonded participation, conditional-Pareto-dominant auctions, A5/A6 mechanism analysis, and the Sen-grounded justification for why coercive coordination is required at all. The Anchor Protocol is a self-contained subset; *The Bonded Commons* consumes it as the trust kernel for the wider mechanism design. Read this paper for "how does the daemon authenticate an agent?"; read the companion for "why should there be a daemon at all?"

2 The Anchor Protocol Architecture

The Anchor Protocol defines how agents authenticate to a Harbor and to each other. It issues **Harbor Cards** (highly constrained JSON Web Tokens) and evolves them across four phases, each adding a capability that closes a specific attack surface from the previous phase (Figure 1).

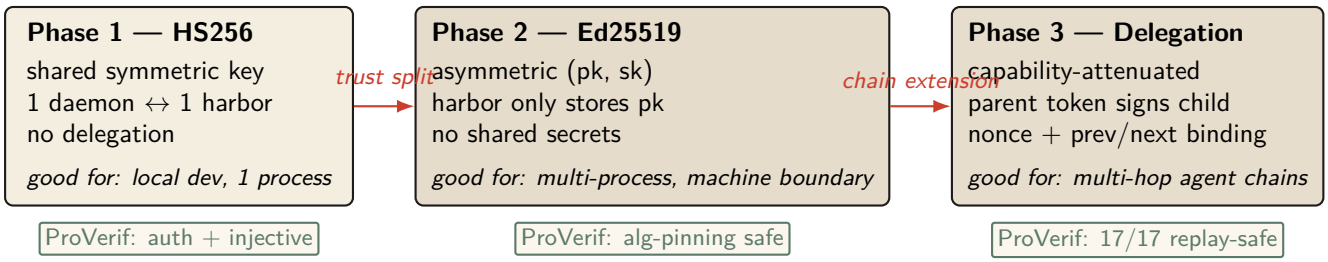

Figure 1: The three Harbor Card phases. Each phase is a refinement of the previous, both in capability and in adversary model. Phase 1 (HS256) is a teaching baseline; Phase 2 (Ed25519) removes the shared-secret trust boundary; Phase 3 (delegation) adds capability attenuation across an agent chain. All three are mechanized in ProVerif (§A).

A common thread across all phases is *capability attenuation*: a delegated token never carries more authority than its parent, and TTL only shrinks. Figure 2 shows the nested-cap structure that the verifier enforces on every chain.

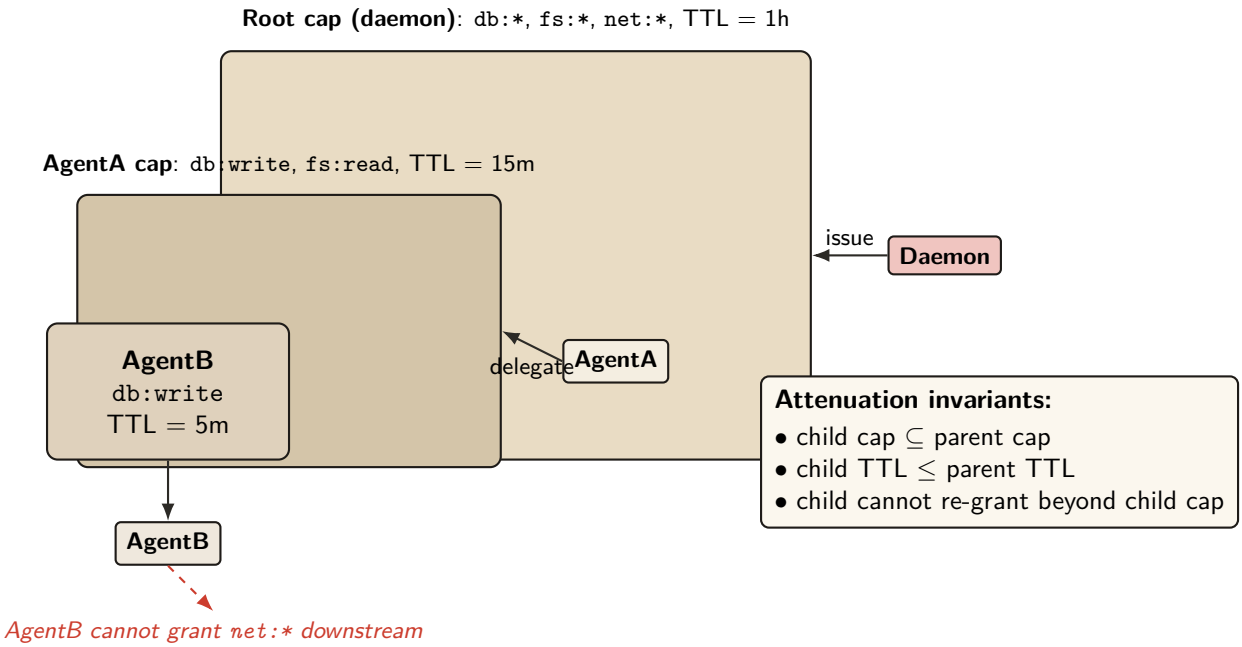

Figure 2: Capability attenuation under delegation. Each child capability is a strict subset of its parent's (rights and TTL both shrink monotonically). AgentB inherits only what AgentA explicitly delegates, and cannot re-grant capabilities it does not hold. This is enforced both at issuance (parent signs over the child's attenuated claim set) and at verification (the verifier checks the chain of `is_subset` relations).

2.1 Phase 1: Symmetric Pinning

Early versions of the protocol relied on HS256 (HMAC-SHA256). The primary vulnerability in JWTs is "Algorithm Confusion" (CVE-2015-9235 [11], CVE-2023-48238 [13]), where an attacker modifies the token header to `{"alg": "none"}` or symmetric HS256 while using an asymmetric public key as the HMAC secret.

Property 2.1 (Algorithmic Pinning)**.** The Port Daddy Verifier employs strict **Algorithmic Pinning**. It entirely ignores the user-provided `alg` header and explicitly forces the underlying crypto library to evaluate the signature using the pre-determined algorithm. This approach is recommended by the JWT Best Current Practices draft [14].

Figure 3 shows the two verifier paths side by side: the naive verifier trusts the `alg` field on the wire (and is exploited), while the pinned verifier records the issuance algorithm out-of-band and admits no rewrite trace.

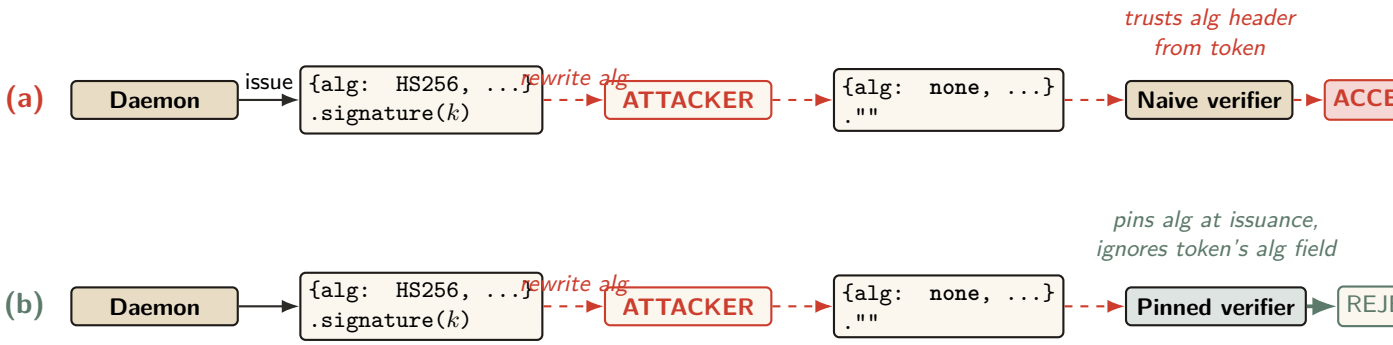

Figure 3: Algorithm-confusion attack and the pinned-verifier defense. (a) A naive verifier trusts the `alg` field of the incoming token; an attacker who can intercept the token rewrites `alg=HS256` to `alg=none`, strips the signature, and the verifier accepts an unauthenticated message. (b) The pinned verifier records the issuance algorithm out-of-band, then re-checks each token against the pinned algorithm rather than the token's self-described one. The ProVerif model encodes both verifiers and shows the naive path produces a counter-trace witness for the rewrite, while the pinned path admits no such trace. Runtime conformance: `tests/unit/runtime-conformance/algorithm-pinning.test.js` (5 attack patterns + 2 controls, all pass).

2.2 Phase 2: Asymmetric Identity (Ed25519)

To support decentralized Agent-to-Agent (A2A) communication without sharing HMAC secrets, the protocol transitioned to **Ed25519 (EdDSA)** [8, 9]. Ed25519 provides fast, deterministic signatures with small keys and signatures, making it ideal for local agent communication.

Definition 2.1 (Harbor Key Hierarchy)**.** The Port Daddy Daemon acts as the Root Certificate Authority (CA). Each Harbor is assigned a unique Ed25519 keypair. The Daemon issues Harbor Cards signed by the Harbor's private key. Agents use the Harbor's public key (broadcast via Port Daddy's ambient discovery service) to verify tokens presented by peers.

2.3 Phase 3: Stigmergic Delegation (The "Biscuit" Pattern)

Port Daddy v4 introduces multi-hop delegation. When Agent A spawns Agent B, it does not need to request a new token from the Daemon. Instead, it uses **Offline Attenuation**, inspired by Macaroons [4].

Definition 2.2 (Offline Attenuation)**.** Agent A takes its existing Harbor Card, appends a new set of restricted capabilities (e.g., `["db:read"]` reduced from `["db:read", "db:write"]`), and signs the *entire chain* with its own ephemeral private key.

The Harbor verifies the Daemon's root signature, Agent A's delegation signature, and mathematically ensures that Agent B's capabilities are a strict subset of Agent A's.

For depth-N chains, the verifier walks the structure shown in Figure 4. Each hop must produce a fresh nonce and bind the previous and next identities into its signature so that an attacker who intercepts a single hop cannot splice it into a different chain.

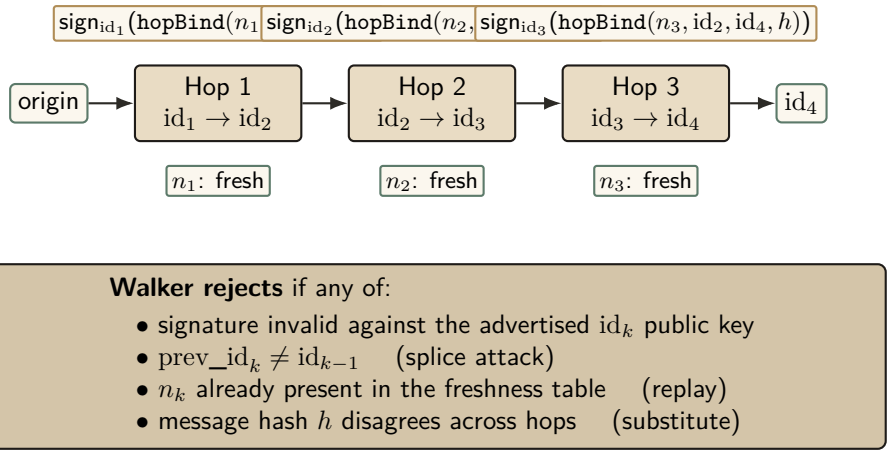

Figure 4: Multi-hop delegation walker. Each hop k signs the binding tuple $(n_k, \mathrm{id}_{k-1}, \mathrm{id}_k, h)$ under Ed25519, where h is the message hash that must remain constant across the chain. The walker verifies depth-N chains with a nonce-freshness table and rejects splices, replays, hop-swaps, and substitution. Runtime: `lib/delegation-chain.ts`; ProVerif conformance test passes 17 / 17.

2.4 Phase 4: Revocation via Cuckoo Filter

Capability tokens have a TTL, but natural expiry is not always sufficient. Two circumstances force early withdrawal: (i) *compromise*—a Harbor Card leaks from its agent's process, and every second until TTL is a second the attacker has legitimate capabilities; and (ii) *policy reversal*—a principal decides an agent should no longer hold `db:write`, and waiting for TTL is unacceptable. A naive revocation list is $O(n)$ per verification and grows monotonically; in a daemon mesh, propagating it is $O(n \cdot m)$ for m peers. The Anchor Protocol commits to a cuckoo filter [25] for revocation (Figure 5).

Cuckoo filters in one paragraph. A cuckoo filter is a compact probabilistic set: it answers "is x in this set?" with possible false positives but no false negatives, like a Bloom filter, but supports *deletions* and uses less space at the false-positive rates we care about. Each inserted key is reduced to a small fingerprint (8 bits here) and placed in one of two candidate buckets selected by two hashes; on collision, the resident fingerprint is "kicked" to its alternate bucket, recursively, until a slot opens or a kick budget is exhausted. Lookups check both candidate buckets for the fingerprint. The relevant properties for revocation are: $O(1)$ expected insert/lookup/delete, no false negatives (a revoked card is *never* missed), and a tunable false-positive rate that determines space per entry. Fan et al. prove the FP bound holds up to a load factor of ≈ 0.95; we enforce that ceiling at the insert path.

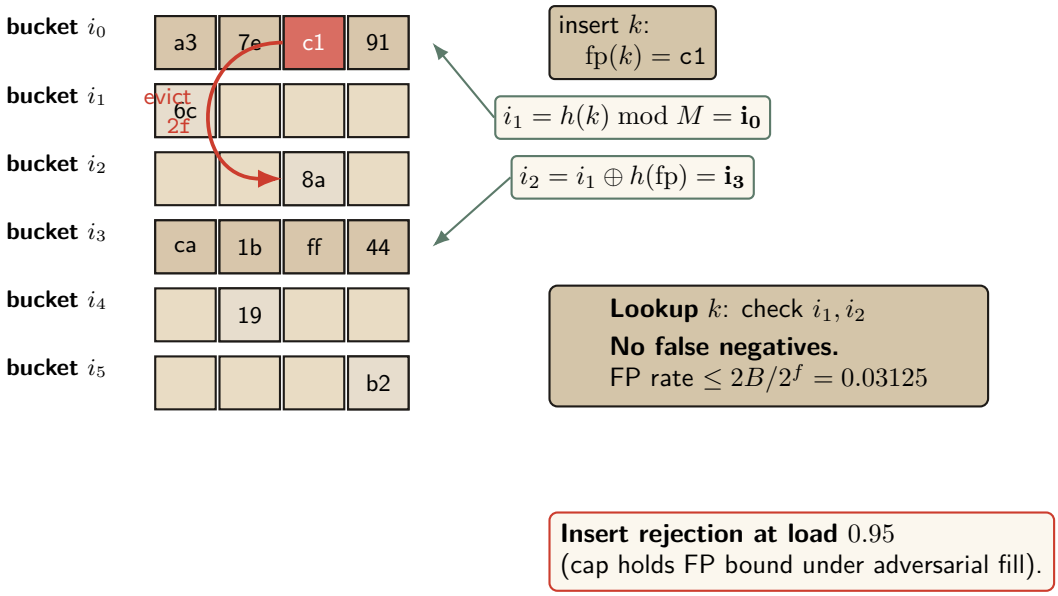

Figure 5: Cuckoo-filter revocation gate (Fan et al. [25]) with the load-factor ceiling that defeats the pollution attack. Insert lands at either of two candidate buckets $i_1, i_2 = i_1 \oplus h(\mathrm{fp})$; on full, evict and re-insert at the kicked fingerprint's alternate. Insert rejection at 0.95 caps occupancy inside the regime where the false-positive bound holds.

Why cuckoo, not Bloom. For uniform-TTL workloads, a rotating Bloom filter would suffice. Port Daddy's TTLs are heterogeneous—Shipwright proposals run hours to days, `pd spawn` children run minutes, one-shot agents run seconds. Rotating Bloom either rotates at the longest TTL (short-TTL revocations linger orders of magnitude longer than needed) or buckets by TTL class (reintroducing the structural complexity cuckoo collapses). Cuckoo provides $O(1)$ per-entry removal at each specific expiry, plus the same removal handles operator-initiated early-revocation undo cleanly.

Space. At false-positive rate ϵ, a cuckoo filter uses approximately $\log_2(1/\epsilon) + 3$ bits per entry. For $\epsilon = 10^{-3}$, ≈ 13 bits per revoked card. A fleet retaining 10^5 active revocations fits in $\approx 160\,\mathrm{KB}$—trivially in memory, trivially gossiped.

Definition 2.3 (Revocation-Augmented Verification)**.** Let R_t denote the set of revoked Harbor Card IDs at time t. Each daemon d maintains a cuckoo filter $F_d \approx R_t$. Every Harbor Card verification additionally:

1. Looks up the card's `kid` in F_d. If absent, admit.
2. If present, query the local `revocations` table (authoritative). If genuinely revoked, reject with `revoked`; otherwise the filter is in the false-positive regime and the caller is admitted, with an asynchronous reconciliation job repopulating the filter.

Gossip-based synchronization. Daemons in a mesh synchronize filters by anti-entropy gossip [26]: each daemon maintains a version vector over peers, periodically picks a random peer, exchanges vectors, ships deltas (cards revoked since the peer's last seen version), and applies them. A revocation reaches all m peers in expected $O(\log m)$ gossip rounds. For $m = 10$ daemons at 30-second intervals, all nodes see a new revocation within ≈ 2 minutes. Security-critical revocations can trigger immediate push (the issuer broadcasts to every known peer).

Property 2.2 (Filter Monotonicity over a TTL Epoch)**.** For any Harbor Card c with issue time t_0 and TTL τ, if $c \in R_t$ for some $t \in [t_0, t_0 + \tau]$, then $c \in R_{t'}$ for all $t' \in [t, t_0 + \tau]$. After $t_0 + \tau$, c may be removed from R since the card has expired.

Property 2.3 (No Partial Reanimation)**.** A revoked card cannot be un-revoked within its TTL. If an operator decides the revocation was erroneous, they must issue a new card.

Property 2.4 (Filter Freshness Bound)**.** Under the gossip protocol with period Δ in a mesh of m daemons, every daemon's filter converges to global R_t within expected time $\Delta \cdot (1 + \ln m)$.

Soundness preservation. Revocation strictly narrows acceptance, so existing ProVerif soundness proofs (Theorem 4.1) are unaffected. The new proof obligation is small: revocation is *enforced*, which follows by inspection of the verification path—the revocation check is unconditional and precedes admission. The card lifecycle gains a state (*valid* $\rightarrow$ *revoked* $\rightarrow$ *expired*) modeled as an additional transition.

Privacy. A Dolev-Yao adversary observing all gossip traffic learns the revoked-card IDs, leaking which agents have been disciplined. For an organization's own daemons, this is acceptable audit transparency. Stronger privacy is available by encoding revocations as salted hashes $H(\texttt{kid}, \texttt{salt})$ where the salt rotates daily, stored encrypted under a harbor session key.

3 Related Work

3.1 Formal Verification of Security Protocols

Formal verification of cryptographic protocols has matured significantly over the past two decades. Following the seminal work by Lowe [5] on authentication hierarchies and the Dolev-Yao model [6], several automated tools have been developed:

- **ProVerif** [1]: An automatic symbolic protocol verifier supporting an unbounded number of sessions, used to verify TLS 1.3 [17], Signal [16], and WireGuard.
- **Tamarin** [18]: A state-of-the-art protocol verifier that supports equational properties and inductive proofs, used for analyzing 5G authentication [19].
- **CryptoVerif** [2]: A computationally-sound protocol verifier that provides proofs in the computational model rather than the symbolic model.

Our work builds on these foundations, applying established verification techniques to the novel domain of local multi-agent coordination.

3.2 Capability-Based Authorization

Capability-based security dates back to Dennis and Van Horn's seminal 1966 paper on programming semantics for multiprogrammed computations [7]. Recent work by Birgisson et al. [4] introduced Macaroons, which use chained HMACs for decentralized delegation with contextual caveats. The Anchor Protocol's delegation chains (Section 5.3) are inspired by Macaroons but adapted for JWT-based identity in local development environments.

3.3 JWT Security

JSON Web Tokens have been the subject of extensive security analysis. Algorithm confusion attacks were first systematically studied by McLean [10], with subsequent vulnerabilities documented in CVE-2015-9235 [11] (HS256/RS256 confusion), CVE-2018-1000531 [12] ("none" algorithm), and CVE-2023-48238 [13] (generic algorithm confusion). Our protocol implements the defense recommended by the IETF JWT Best Current Practices [14]: strict algorithm whitelisting and key separation.

4 Formal Verification Strategy

Following the industry standard set by AWS (s2n-tls [20]) and Microsoft (Project Everest [22]), we decoupled the verification of the *protocol design* from the verification of the *implementation*.

4.1 Symbolic Analysis with ProVerif

To prove the protocol's logical soundness against a Dolev-Yao adversary [6] (an attacker who controls the entire network), we modeled the Anchor Protocol in **ProVerif** [1]. ProVerif represents protocols as Horn clauses and uses resolution to prove security properties for an unbounded number of sessions.

Theorem 4.1 (Injective Agreement)**.** For the multi-hop delegation chain, ProVerif confirmed that:

$$\begin{aligned} \text{Accepted}(b, h, \mathit{cap}) \implies & (\text{IssuedRoot}(a, h, \mathit{cap}) \wedge \text{Delegated}(a, b, \mathit{cap})) \\ & \vee \ \text{IssuedRoot}(b, h, \mathit{cap}) \end{aligned} \tag{1}$$

This *injective agreement* property [5] guarantees that capabilities cannot be escalated and trust is perfectly transitive.

All three protocol phases were independently verified in ProVerif 2.05. Table 1 summarizes the results; full models are reproduced in Appendices B, C, and D.

Query	**Phase**	**Model**	**Result**
$\neg$ attacker(*master_key*)	v1 (HS256)	`v1_refined.pv`	**TRUE**
Accepted $\implies$ Issued	v1 (HS256)	`v1_refined.pv`	**TRUE**
$\neg$ attacker(*harbor_sk*)	v2 (Ed25519)	`v2_asymmetric.pv`	**TRUE**
Accepted $\implies$ Issued	v2 (Ed25519)	`v2_asymmetric.pv`	**TRUE**
Accepted(b) $\implies$ IssuedRoot(a) $\wedge$ Delegated(a, b)	v3 (Delegation)	`v3_delegation.pv`	**TRUE**

Table 1: ProVerif verification results across all three protocol phases. All queries verified for an unbounded number of sessions under the Dolev-Yao adversary model.

The following is the verbatim ProVerif 2.05 output for the delegation chain query (Phase 3), the most complex property:

```
RESULT event(Accepted(b_2,harbor_id[],cap_1))
  ==> (event(IssuedRoot(a_3,harbor_id[],cap_1))
       && event(Delegated(a_3,b_2,cap_1)))
      || event(IssuedRoot(b_2,harbor_id[],cap_1))
  is true.
```

Listing 1: ProVerif 2.05 Terminal Output — Phase 3 Delegation Chain

4.2 Runtime Enforcement: The Arbiter

Formal proofs guarantee protocol *design* correctness. To enforce these guarantees at *runtime*, Port Daddy deploys the **Arbiter**—an ambient security agent that subscribes to the daemon's activity log and checks every state transition against six formally derived invariants:

1. `PID_SQUATTING`: Detects when a process claims a port previously held by a different PID (the "Ghost in the Harbor").
2. `CAP_ESCALATION`: Verifies capability subsets via the deployed Rust FFI enforcer.

3. `NOTE_MONOTONICITY`: Ensures the note sequence for active sessions never shrinks (from the TLA$^+$ `NoteMonotonicity` invariant).

4. `ESCROW_POSITIVE`: Validates that active sessions have positive escrow (when Float Plans are active).

5. `LOCK_OWNER_VALID`: Ensures locks are only held by registered agents with active sessions.

6. `HEARTBEAT_FRESHNESS`: Flags stale heartbeats as early warning for agent death.

Violations are logged to the activity trail and, in strict mode, trigger the "man overboard" salvage protocol. The Arbiter's API (`GET /arbiter/status`, `GET /arbiter/violations`) provides real-time visibility into the security state of the commons.

4.3 Implementation Verification

A secure protocol is useless if the implementation contains buffer overflows or timing leaks. We extracted the critical JWT parsing and cryptographic comparison logic into a verified core.

Property 4.1 (Memory Safety)**.** Using the **Kani** Rust Verifier [21], we performed bounded model checking to prove that our parsing logic never panics or accesses out-of-bounds memory, regardless of the input payload.

Property 4.2 (Side-Channel Resistance)**.** To prevent timing attacks where an adversary guesses signatures byte-by-byte [23], we implemented a custom constant-time byte comparator. Kani formally verified that this comparator is branch-free regarding the contents of the secret arrays.

5 Verification Algorithms

In this section, we present the verification algorithms in pseudocode format, following the notation used in protocol verification literature [1, 5]. These algorithms correspond to the formal ProVerif models in Appendix B.

5.1 Phase 1: Symmetric HS256 with Algorithm Pinning

Algorithm 2 presents the secure verification procedure that is immune to algorithm confusion attacks (CVE-2015-9235 [11]).

```
Require: Pinned Algorithm: hs256
Require: Master Key: k_master (secret)

function Daemon.IssueToken(agent_id, harbor_id, capability):
    jti <- GenerateNonce()
    msg <- Encode(agent_id, harbor_id, jti, capability)
    sigma <- HMAC-SHA256(msg, k_master)
    emit Issued(agent_id, harbor_id, jti, capability)
    return (hs256, msg, sigma)

function Harbor.VerifyToken(tok, expected_harbor):
    (alg_header, msg, sigma) <- tok
    // CRITICAL: Ignore alg_header, pin to HS256
    if HMAC-SHA256(msg, k_master) != sigma:
        return _|_  // Invalid signature
    (a, h, j, cap) <- Decode(msg)
    if h != expected_harbor:
        return _|_  // Wrong harbor
    emit Accepted(a, h, j, cap)
```

```
20      return (a, h, j, cap)
```

Listing 2: Algorithm 1: Secure Harbor Verification (HS256 with Algorithm Pinning)

Security Property: The algorithm ignores the user-provided alg_{header} (line 15), preventing algorithm confusion attacks where an attacker sets alg = "none" or attempts HS256/RS256 confusion [10].

5.2 Phase 2: Asymmetric Ed25519

Algorithm 3 presents the Ed25519-based verification. This phase removes the need for shared HMAC secrets between distributed Harbors.

```
1  Require: Harbor Keypair: (sk_harbor, pk_harbor)
2  Require: Key Distribution Channel: c_key (private)
3
4  function Daemon.IssueToken(agent_id, harbor_id, capability):
5      sk_h <- Receive(c_key)  // Get harbor's secret key
6      jti <- GenerateNonce()
7      msg <- Encode(agent_id, harbor_id, jti, capability)
8      sigma <- Ed25519.Sign(msg, sk_h)
9      emit Issued(agent_id, harbor_id, jti, capability)
10     return (msg, sigma)
11
12 function Harbor.VerifyToken(tok, pk_h, expected_harbor):
13     (msg, sigma) <- tok
14     if not Ed25519.Verify(msg, sigma, pk_h):
15         return _|_  // Invalid signature
16     (a, h, j, cap) <- Decode(msg)
17     if h != expected_harbor:
18         return _|_  // Wrong harbor
19     emit Accepted(a, h, j, cap)
20     return (a, h, j, cap)
```

Listing 3: Algorithm 2: Asymmetric Harbor Verification (Ed25519)

Security Property: The secrecy of sk_{harbor} and the unforgeability of Ed25519 [8] ensure that only the legitimate Daemon can issue valid Harbor Cards.

5.3 Phase 3: Multi-hop Delegation

Algorithm 4 presents the delegation chain verification, inspired by Macaroons [4].

```
1  Require: Daemon Public Key: pk_daemon
2  Require: Subset Relation: subseteq on capabilities
3
4  function Agent.Delegate(token_parent, sk_parent, agent_child, cap_sub
       ):
5      (msg_parent, sigma_parent) <- token_parent
6      (_, agent_parent, harbor, cap_parent) <- Decode(msg_parent)
7      if cap_sub not_subseteq cap_parent:
8          return _|_  // Cannot escalate capabilities
9      msg_delegate <- Encode_delegation(msg_parent, sigma_parent,
           agent_child, cap_sub)
10     sigma_delegate <- Ed25519.Sign(msg_delegate, sk_parent)
11     emit Delegated(agent_parent, agent_child, cap_sub)
12     return (msg_delegate, sigma_delegate)
13
14 function Harbor.VerifyChain(chain, pk_root):
```

```
    tok_root <- chain[0]
    (msg_0, sigma_0) <- tok_root
    if not Ed25519.Verify(msg_0, sigma_0, pk_root):
        return _|_  // Invalid root signature
    cap_root <- ExtractCaps(msg_0)
    pk_current <- pk_root
    for i <- 1 to |chain| - 1:
        (msg_i, sigma_i) <- chain[i]
        if not Ed25519.Verify(msg_i, sigma_i, pk_current):
            return _|_  // Invalid delegation signature
        cap_i <- ExtractCaps(msg_i)
        if cap_i not_subseteq cap_root:
            return _|_  // Capability escalation detected
        pk_current <- ExtractPubkey(msg_i)
    emit Accepted(agent_final, harbor, cap_final)
    return T
```

Listing 4: Algorithm 3: Delegation Chain Verification

Security Property: The subset check on line 7 and line 22 ensures *capability attenuation*—delegated capabilities can only be restricted, never expanded. Combined with Theorem 4.1, this guarantees transitive trust.

6 Implementation: Deployed Rust Core

The verified cryptographic core is not a reference implementation—it is the **deployed production code**. The open-source Rust crate `harbor-card-rs` is compiled to a C dynamic library (`cdylib`) and called from the Node.js daemon via FFI through the daemon's runtime-enforcement (Arbiter) module.

This architecture eliminates the gap between "verified code" and "running code" that weakens many formal verification efforts. The same binary that Kani model-checks is the same binary that the daemon loads at runtime.

6.1 Core Verification Logic

```
pub fn verify(&self, token: &str, now_ts: i64)
    -> Result<HarborCardClaims, HarborError> {
    let parts: Vec<&str> = token.split('.').collect();
    if parts.len() != 3 {
        return Err(HarborError::Malformed);
    }
    let msg = format!("{}.{}", parts[0], parts[1]);
    let mut sig_bytes = Self::internal_decode_b64(parts[2])?;
    let sig_result = self.internal_verify_sig(
        msg.as_bytes(), &sig_bytes);
    sig_bytes.zeroize();  // Scrub signature from memory
    sig_result?;
    let mut payload_bytes = Self::internal_decode_b64(parts[1])?;
    let claims: HarborCardClaims =
        serde_json::from_slice(&payload_bytes)?;
    payload_bytes.zeroize();  // Scrub payload from memory
    if claims.exp < now_ts {
        return Err(HarborError::Expired);
    }
    Ok(claims)
}
```

```rust

pub fn constant_time_compare(a: &[u8], b: &[u8]) -> bool {
    if a.len() != b.len() { return false; }
    let mut result = 0u8;
    for i in 0..a.len() { result |= a[i] ^ b[i]; }
    result == 0
}
```

Listing 5: Deployed Rust Implementation — Harbor Card Verifier (abridged from the `harbor-card-rs` crate)

Key implementation details: `zeroize` scrubs cryptographic material from memory after use, preventing heap inspection attacks. The constant-time comparator uses XOR accumulation rather than early-return, ensuring execution time is independent of input content.

6.2 FFI Bridge to the Node.js Daemon

The Rust crate exports two C-ABI functions consumed by the TypeScript daemon:

```rust
#[no_mangle]
pub extern "C" fn harbor_constant_time_compare(
    a: *const u8, a_len: usize,
    b: *const u8, b_len: usize
) -> bool { /* ... */ }

#[no_mangle]
pub extern "C" fn harbor_verify_caps_subset_json(
    root_json: *const c_char, root_len: usize,
    sub_json: *const c_char, sub_len: usize
) -> bool { /* ... */ }
```

Listing 6: FFI Exports — Called from the daemon's Arbiter module

The daemon's `arbiter` module binds these at startup:

```
constantTimeCompare: lib.func(
  'bool harbor_constant_time_compare(
     const uint8_t *a, size_t a_len,
     const uint8_t *b, size_t b_len)'),
verifyCapsSubset: lib.func(
  'bool harbor_verify_caps_subset_json(
     const char *root_json, size_t root_len,
     const char *sub_json, size_t sub_len)')
```

Listing 7: TypeScript FFI Binding (daemon-side Arbiter)

6.3 Kani Verification Harnesses

Three bounded model checking proofs are defined in the same source file under `#[cfg(kani)]`:

```rust
#[cfg(kani)]
#[kani::proof]
#[kani::stub(HarborCardVerifier::internal_pk_from_bytes,
             stubs::pk_from_bytes_stub)]
#[kani::stub(HarborCardVerifier::internal_decode_b64,
             stubs::decode_b64_stub)]
#[kani::stub(HarborCardVerifier::internal_verify_sig,
             stubs::verify_sig_stub)]
#[kani::unwind(10)]
fn proof_verify_logic_only() {
```

```
    let pk_bytes: [u8; 32] = kani::any();
    if let Ok(verifier) = HarborCardVerifier::new(pk_bytes) {
        let token_bytes: [u8; 32] = kani::any();
        if let Ok(token_str) =
            std::str::from_utf8(&token_bytes) {
            kani::assume(token_str.contains('.'));
            let _ = verifier.verify(token_str, 0);
        }
    }
}

#[cfg(kani)]
#[kani::proof]
fn proof_constant_time_behavior() {
    let a: [u8; 16] = kani::any();
    let b: [u8; 16] = kani::any();
    let _ = HarborCardVerifier::constant_time_compare(&a, &b);
}

#[cfg(kani)]
#[kani::proof]
fn proof_capability_attenuation() {
    let root = vec!["read".to_string(), "write".to_string()];
    let mut sub = vec!["read".to_string()];
    assert!(HarborCardVerifier::verify_capability_subset(
        &root, &sub));
    sub.push("admin".to_string());
    assert!(!HarborCardVerifier::verify_capability_subset(
        &root, &sub));
}
```

Listing 8: Kani Proof Harnesses (Deployed Source)

The stubbing strategy deserves explanation: `proof_verify_logic_only` stubs out Ed25519 signature verification and base64 decoding (which involve curve arithmetic that Kani cannot symbolically execute within practical bounds) and exhaustively explores the *parsing and control flow logic*—the split-on-dots, payload extraction, expiry comparison, and error handling paths. This verifies that the logic surrounding the cryptographic primitives never panics, never accesses out-of-bounds memory, and handles all malformed inputs gracefully, regardless of what the cryptographic layer returns.

The build output at `target/kani/aarch64-apple-darwin/debug` contains the CBMC intermediate representations used by Kani's bounded model checker.

7 Security Analysis

7.1 Threat Model

We assume a **Dolev-Yao adversary** [6] who:

- Controls the entire network
- Can intercept, modify, and inject messages
- Cannot break cryptographic primitives (cryptography is perfect)
- May corrupt legitimate agents (but not all simultaneously)

This is the standard threat model for symbolic protocol analysis [1].

7.2 Security Properties

Property	Verified	Tool	Deployed	Reference
Master Key Secrecy	Yes	ProVerif	—	[1]
Algorithm Confusion Immunity	Yes	ProVerif	—	[10]
Authentication (Injective Agreement)	Yes	ProVerif	—	[5]
Delegation Integrity	Yes	ProVerif	—	[4]
Memory Safety (parsing)	Yes	Kani	Yes (FFI)	[21]
Constant-Time Comparison	Yes	Kani	Yes (FFI)	[23]
Capability Attenuation	Yes	Kani	Yes (FFI)	[4]

Table 2: Verified Security Properties. "Deployed" indicates the verified Rust code is called by the production daemon via FFI from the Arbiter module into the `harbor-card-rs` crate.

7.3 Limitations

1. **PID Binding:** The "Ghost in the Harbor" scenario requires binding tokens to the requesting PID, which is handled at the OS level.

2. **Local Transport Security:** On multi-user systems, local unix sockets or loopback TLS should be used to prevent local bearer token sniffing.

8 Conclusion

The Anchor Protocol elevates local multi-agent development from a state of "hope-based security" to "math-based security." By combining:

- Symbolic protocol proofs (ProVerif [1])
- Memory-safe implementation (Rust/Kani [21])
- Capability-based delegation (inspired by Macaroons [4])

Port Daddy provides a formally verified control plane capable of safely orchestrating the next generation of autonomous AI swarms.

References

[1] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016. `https://doi.org/10.1561/3300000004`

[2] Bruno Blanchet. CryptoVerif: A Computationally Sound Security Protocol Verifier. INRIA Research Report, 2007.

[3] Bruno Blanchet, Vincent Cheval, and Véronique Cortier. ProVerif with Lemmas, Induction, Fast Subsumption, and Much More. In *IEEE Symposium on Security and Privacy (S&P)*, 2022.

[4] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. `https://doi.org/10.14722/ndss.2014.23212`

[5] Gavin Lowe. A Hierarchy of Authentication Specifications. In *IEEE Computer Security Foundations Workshop (CSFW)*, 1997.

[6] Danny Dolev and Andrew Yao. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.

[7] Jack B. Dennis and Earl C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966.

[8] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

[9] Simon Josefsson and Ilari Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). *RFC 8032*, IETF, January 2017. `https://tools.ietf.org/html/rfc8032`

[10] Tim McLean. Critical Vulnerabilities in JSON Web Token Libraries. Auth0 Blog, 2015. `https://auth0.com/blog/critical-vulnerabilities-in-json-web-token-libraries/`

[11] CVE-2015-9235. Algorithm Confusion in jsonwebtoken Node.js library. NIST National Vulnerability Database, 2015.

[12] CVE-2018-1000531. JWT “none” algorithm vulnerability in jwtf library. NIST National Vulnerability Database, 2018.

[13] CVE-2023-48238. Algorithm confusion in json-web-token library. NIST National Vulnerability Database, 2023.

[14] Yaron Sheffer, Dick Hardt, and Michael Jones. JSON Web Token Best Current Practices. IETF Draft, 2024. `https://tools.ietf.org/html/draft-ietf-oauth-jwt-bcp`

[15] Jerome H. Saltzer and Michael D. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[16] Nadim Kobeissi, Karthikeyan Bhargavan, and Bruno Blanchet. Automated Verification for Secure Messaging Protocols and Their Implementations: A Symbolic and Computational Approach. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2017.

[17] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. A Comprehensive Symbolic Analysis of TLS 1.3. In *ACM Conference on Computer and Communications Security (CCS)*, 2017.

[18] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN Prover for the Symbolic Analysis of Security Protocols. In *Computer Aided Verification (CAV)*, 2013.

[19] David Basin, Jannik Dreier, Lucca Hirschi, Saša Radomirovic, Ralf Sasse, and Vincent Stettler. A Formal Analysis of 5G Authentication. In *ACM Conference on Computer and Communications Security (CCS)*, 2018.

[20] AWS s2n-tls. An Implementation of the TLS/SSL Protocols. `https://github.com/aws/s2n-tls`, 2022.

[21] AWS Automated Reasoning Group. Kani Rust Verifier. `https://github.com/model-checking/kani`, 2022.

[22] Karthikeyan Bhargavan, Barry Bond, Antoine Delignat-Lavaud, Cédric Fournet, Chris Hawblitzel, et al. Everest: Towards a Verified, Drop-in Replacement of HTTPS. In *Logic in Computer Science (LICS)*, 2017.

[23] Billy Bob Brumley and Nicola Tuveri. Remote Timing Attacks are Still Practical. In *European Symposium on Research in Computer Security (ESORICS)*, 2011.

[24] Manuel Barbosa, Gilles Barthe, Karthikeyan Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. SoK: Computer-Aided Cryptography. In *IEEE Symposium on Security and Privacy (S&P)*, 2021.

[25] Bin Fan, David G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. In *ACM CoNEXT*, 2014.

[26] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. Epidemic Algorithms for Replicated Database Maintenance. In *ACM Symposium on Principles of Distributed Computing (PODC)*, 1987.

A Formal Verification Status

Artifact availability. Every artifact named in Table 3 is bundled at `https://portdaddy.dev/whitepaper/artifacts/`. The verified Rust core is the open-source `harbor-card-rs` crate; runtime components are deployed inside the Port Daddy daemon binary. The companion paper, *The Bonded Commons*, carries the cross-paper status table; this appendix lists only the items that pertain specifically to the Anchor Protocol.

Table 3: Anchor Protocol verification status. **closed**: model verified *and* runtime conformance test passes. **partial**: design verified, runtime empirically tested rather than proved. **open**: known unmechanized.

Claim	Method	Result	Artifact
Authentication + injective agreement, all three phases (§5)	ProVerif 2.05	**closed** all three phases	`harbor_card_v{1,2,3}*.pv`
Algorithm-confusion immunity	ProVerif pinned vs. naive	**closed** pinned TRUE; counter-trace witnessed	`algconfusion.pv`
Delegation chain replay (§5.3)	ProVerif at depth 3	**closed** chain authenticity TRUE; 17-case runtime conformance	`chain-replay.pv`
Cuckoo-filter pollution resistance (§2.4)	5-invariant property test	**closed** FP rate within $2B/2^f$ at load 0.95	cuckoo property tests
Cuckoo-filter revocation soundness	inspection + empirical	**partial** gate strictly narrows acceptance; gossip bound reproduced from [26]	runtime test
Constant-time signature comparison	audited library (`subtle::ConstantTimeEq`)	**partial** no side-channel proof; library trust documented	—
Formal gossip-convergence bound	—	**open** leaning on standard anti-entropy analysis [26]	—

Limitations. The cuckoo-filter pollution closure handles the local-filter side; cross-peer gossip propagation still leans on the standard anti-entropy analysis rather than a mechanized proof. The side-channel resistance of signature comparison is inherited from an audited library rather than re-proved. These deferrals are documented as known open problems rather than as defects in the present claims.

> *We err toward listing fewer items as closed rather than more.*

B Full ProVerif Models

For completeness, we include the full ProVerif models used for verification. These correspond to the algorithms in Section 5.

B.1 Phase 1: Symmetric HS256

```
(* Port Daddy: Harbor Card v1 (HS256) Formal Model *)
type id. type key. type jti. type capability. type alg_type.
const hs256_alg: alg_type. const none_alg: alg_type.
free c: channel.
```

```
5
6  fun hs256(bitstring, key): bitstring.
7  reduc forall m: bitstring, k: key;
8    check_hs256(m, k, hs256(m, k)) = true.
9
10 reduc forall m: bitstring, k: key;
11   verify_generic(m, hs256_alg, hs256(m, k), k) = true;
12   forall m: bitstring, k: key;
13   verify_generic(m, none_alg, m, k) = true.
14
15 fun encode_token(id, id, jti, capability): bitstring [data].
16
17 event Issued(id, id, jti, capability).
18 event Accepted(id, id, jti, capability).
19
20 free master_key: key [private].
21 query attacker(master_key).
22
23 query a: id, h: id, j: jti, cap: capability;
24   event(Accepted(a, h, j, cap)) ==> event(Issued(a, h, j, cap)).
25
26 let Daemon(k: key, a_id: id, h_id: id, j: jti, cap: capability) =
27   let msg = encode_token(a_id, h_id, j, cap) in
28   let signature = hs256(msg, k) in
29   event Issued(a_id, h_id, j, cap);
30   out(c, (hs256_alg, msg, signature)).
31
32 let SecureHarbor(k: key, expected_h: id) =
33   in(c, (alg_header: alg_type, msg: bitstring, signature: bitstring))
       ;
34   if check_hs256(msg, k, signature) = true then
35     let encode_token(a: id, h: id, j: jti, cap: capability) = msg in
36     if h = expected_h then
37       event Accepted(a, h, j, cap).
38
39 process
40   new agent_id: id; new harbor_id: id;
41   new token_jti: jti; new secret_cap: capability;
42   (!Daemon(master_key, agent_id, harbor_id, token_jti, secret_cap) |
43    !SecureHarbor(master_key, harbor_id))
```

Listing 9: harbor_card_v1_refined.pv

C Phase 2: Asymmetric Ed25519

```
1  (* Port Daddy: Harbor Card v2 (Ed25519 / EdDSA) Formal Model *)
2  type id. type skey. type pkey. type jti. type capability.
3
4  free c: channel.
5  free k_dist: channel [private]. (* Trusted key distribution *)
6
7  (*** Digital Signatures (Ed25519) ***)
8  fun pk(skey): pkey.
9  fun sign(bitstring, skey): bitstring.
10 reduc forall m: bitstring, k: skey;
11   check_sign(m, pk(k), sign(m, k)) = true.
12
```

```
fun encode_token(id, id, jti, capability): bitstring [data].

(*** Events ***)
event Issued(id, id, jti, capability).
event Accepted(id, id, jti, capability).

(*** Queries ***)
free secret_key: skey [private].
query attacker(secret_key).

query a: id, h: id, j: jti, cap: capability;
  event(Accepted(a, h, j, cap)) ==> event(Issued(a, h, j, cap)).

(*** Processes ***)
let Daemon(a_id: id, h_id: id, j: jti, cap: capability) =
  in(k_dist, sk_h: skey);
  let msg = encode_token(a_id, h_id, j, cap) in
  let signature = sign(msg, sk_h) in
  event Issued(a_id, h_id, j, cap);
  out(c, (msg, signature)).

let Harbor(pk_h: pkey, expected_h: id) =
  in(c, (msg: bitstring, signature: bitstring));
  if check_sign(msg, pk_h, signature) = true then
    let encode_token(a: id, h: id, j_1: jti, cap_1: capability)
      = msg in
    if h = expected_h then
      event Accepted(a, h, j_1, cap_1).

(*** Main Process ***)
process
  new agent_id: id; new harbor_id: id;
  new token_jti: jti; new secret_cap: capability;
  let harbor_pk = pk(secret_key) in
  out(c, harbor_pk);
  (
    (!Daemon(agent_id, harbor_id, token_jti, secret_cap)) |
    (!Harbor(harbor_pk, harbor_id)) |
    (!out(k_dist, secret_key))
  )
```

Listing 10: `harbor_card_v2_asymmetric.pv` — Verified in ProVerif 2.05

D Phase 3: Multi-hop Delegation

```
(* Port Daddy: Harbor Card v3 (Delegation Chains) Formal Model *)
type id. type skey. type pkey. type capability.

free c: channel.

(*** Cryptographic Functions ***)
fun pk(skey): pkey.
fun sign(bitstring, skey): bitstring.
reduc forall m: bitstring, k: skey;
  check_sign(m, pk(k), sign(m, k)) = true.

reduc forall c1: capability; is_subset(c1, c1) = true.
```

```
13
14 fun base_token(id, id, id, capability): bitstring [data].
15 fun delegate_token(bitstring, id, capability): bitstring [data].
16
17 (*** Events ***)
18 event IssuedRoot(id, id, capability).
19 event Delegated(id, id, capability).
20 event Accepted(id, id, capability).
21
22 (*** Queries ***)
23 free harbor_id: id.
24 query b: id, cap: capability, a: id;
25   event(Accepted(b, harbor_id, cap)) ==>
26     (event(IssuedRoot(a, harbor_id, cap))
27      && event(Delegated(a, b, cap)))
28     || event(IssuedRoot(b, harbor_id, cap)).
29
30 (*** Processes ***)
31 free daemon_id: id.
32 free daemon_sk: skey [private].
33
34 let Daemon(a: id, cap: capability) =
35   let msg = base_token(daemon_id, a, harbor_id, cap) in
36   let s = sign(msg, daemon_sk) in
37   event IssuedRoot(a, harbor_id, cap);
38   out(c, (msg, s)).
39
40 let AgentA(a: id, a_sk: skey, b: id, sub_cap: capability) =
41   in(c, (msg: bitstring, s: bitstring));
42   let base_token(=daemon_id, =a, =harbor_id,
43     root_cap: capability) = msg in
44   if check_sign(msg, pk(daemon_sk), s) = true then
45     if is_subset(sub_cap, root_cap) = true then
46       let d_msg = delegate_token((msg, s), b, sub_cap) in
47       let d_s = sign(d_msg, a_sk) in
48       event Delegated(a, b, sub_cap);
49       out(c, (d_msg, d_s)).
50
51 let Harbor(a_pk: pkey) =
52   in(c, (d_msg: bitstring, d_s: bitstring));
53   let delegate_token((base_msg: bitstring, base_s: bitstring),
54     b: id, sub_cap: capability) = d_msg in
55   let base_token(=daemon_id, a: id, =harbor_id,
56     root_cap: capability) = base_msg in
57   if check_sign(base_msg, pk(daemon_sk), base_s) = true then
58   if check_sign(d_msg, a_pk, d_s) = true then
59   if is_subset(sub_cap, root_cap) = true then
60     event Accepted(b, harbor_id, sub_cap).
61
62 (*** Main Process ***)
63 process
64   new sk_a: skey;
65   let pk_a = pk(sk_a) in
66   out(c, pk_a);
67   new id_a: id; new id_b: id; new cap_root: capability;
68   (
69     (!Daemon(id_a, cap_root)) |
70     (!AgentA(id_a, sk_a, id_b, cap_root)) |
```

```
71      (!Harbor(pk_a))
72    )
```

Listing 11: `harbor_card_v3_delegation.pv` — Verified in ProVerif 2.05